

Fonctions JavaScript

Dans ce chapitre, vous apprendrez à définir et à appeler une fonction en JavaScript.

Qu'est-ce que la fonction ?

Une fonction est un groupe d'instructions qui exécutent des tâches spécifiques et peuvent être conservées et maintenues séparément du programme principal. Les fonctions permettent de créer des packages de code réutilisables qui sont plus portables et plus faciles à déboguer. Voici quelques avantages de l'utilisation des fonctions :

- **Une fonction réduit la répétition de code dans un programme** — Une fonction vous permet d'extraire un bloc de code couramment utilisé en un seul composant. Vous pouvez maintenant effectuer la même tâche en appelant cette fonction où vous voulez dans votre script sans avoir à copier et coller le même bloc de code encore et encore.
- **Les fonctions rendent le code beaucoup plus facile à maintenir** - Puisqu'une fonction créée une fois peut être utilisée plusieurs fois, toute modification apportée à l'intérieur d'une fonction est automatiquement implémentée à tous les endroits sans toucher aux différents fichiers.
- **Les fonctions facilitent l'élimination des erreurs** - Lorsque le programme est subdivisé en fonctions, si une erreur se produit, vous savez exactement quelle fonction est à l'origine de l'erreur et où la trouver. Par conséquent, la correction des erreurs devient beaucoup plus facile.

La section suivante vous montrera comment définir et appeler des fonctions dans vos scripts.

Définir et appeler une fonction

La déclaration d'une fonction commence par le mot-clé **function**, suivi du nom de la fonction que vous souhaitez créer, suivi de parenthèses et enfin placez le code de votre fonction entre accolades {}. Voici la syntaxe de base pour déclarer une fonction

```
function nom_fonction() {  
    // Code à exécuter  
}
```

Voici un exemple simple d'une fonction, qui affichera un message bonjour :

```
// Définir la fonction  
function sayHello() {  
    document.write("Hello, bienvenue dans mon site !");  
}  
  
// Appel de la fonction  
sayHello(); // Prints: Hello, bienvenue dans mon site !
```

Une fois qu'une fonction est définie, elle peut être appelée de n'importe où dans le document, en tapant son nom suivi des parenthèses, comme sayHello() dans l'exemple ci-dessus.

Remarque : Un nom de fonction doit commencer par une lettre ou un caractère de soulignement et non par un chiffre, éventuellement suivi par d'autres lettres, chiffres ou caractères de soulignement. Les noms de fonctions sont sensibles à la casse, tout comme les noms de variables.

Ajout de paramètres aux fonctions

Vous pouvez spécifier des paramètres lorsque vous définissez votre fonction pour accepter des valeurs d'entrée au moment de l'exécution. Les paramètres fonctionnent comme des variables d'espace réservé dans une fonction ; ils sont remplacés au moment de l'exécution par les valeurs (appelées arguments) fournies à la fonction au moment de l'invocation.

Les paramètres sont définis sur la première ligne de la fonction à l'intérieur du jeu de parenthèses, comme ceci

```
function nom_fonction( paramètre1 , paramètre2 , paramètre3 ) {  
    // Code à exécuter  
}
```

La fonction `displaySum()` dans l'exemple suivant prend deux nombres comme arguments, il suffit de les additionner, puis d'afficher le résultat dans le navigateur.

```
// Defining function
function displaySum(num1, num2) {
    let total = num1 + num2;
    document.write(total);
}

// Calling function
displaySum(6, 20); // Prints: 26
document.write("<br>");
displaySum(-5, 17); // Prints: 12
```

Vous pouvez définir autant de paramètres que vous le souhaitez. Cependant, pour chaque paramètre que vous spécifiez, un argument correspondant doit être passé à la fonction lorsqu'elle est appelée, sinon sa valeur devient undefined. Considérons l'exemple suivant

```
// Defining function
function showFullname(firstName, lastName) {
    document.write(firstName + " " + lastName);
}

// Calling function
showFullname("Clark", "Kent"); // Prints: Clark Kent
document.write("<br>");
showFullname("John"); // Prints: John undefined
```

Valeurs par défaut des paramètres de fonction

Avec ES6, vous pouvez désormais spécifier des valeurs par défaut pour les paramètres de la fonction. Cela signifie que si aucun argument n'est fourni à la fonction lorsqu'elle est appelée, ces valeurs de paramètres par défaut seront utilisées. C'est l'une des fonctionnalités les plus attendues de JavaScript. Voici un exemple :

```
function myFunction(x, y = 2) {  
  return x * y;  
}  
document.getElementById("demo").innerHTML = myFunction(4);
```

Renvoyer des valeurs à partir d'une fonction

Une fonction peut renvoyer une valeur au script qui a appelé la fonction en conséquence à l'aide de l'instruction **return**. La valeur peut être de n'importe quel type, y compris des tableaux et des objets. L'instruction **return** est généralement placée sur la dernière ligne de la fonction avant l'accolade fermante et la termine par un point-virgule, comme illustré dans l'exemple suivant.

```
function getSum(num1, num2) {  
    let total = num1 + num2;  
    return total;  
}  
  
// Displaying returned value  
document.write(getSum(6, 20) + "<br>"); // Prints: 26  
document.write(getSum(-5, 17)); // Prints: 12
```

Une fonction ne peut pas renvoyer plusieurs valeurs. Cependant, vous pouvez obtenir des résultats similaires en renvoyant un tableau de valeurs, comme illustré dans l'exemple suivant

```
// Defining function
function divideNumbers(dividend, divisor){
    let quotient = dividend / divisor;
    let arr = [dividend, divisor, quotient];
    return arr;
}

// Store returned value in a variable
let all = divideNumbers(10, 2);

// Displaying individual values
document.write(all[0] + "<br>"); // Prints: 10
document.write(all[1] + "<br>"); // Prints: 2
document.write(all[2]); // Prints: 5
```


Travailler avec des expressions de fonction

La syntaxe que nous avons utilisée auparavant pour créer des fonctions est appelée déclaration de fonction. Il existe une autre syntaxe pour créer une fonction appelée expression de fonction.

```
// Declaration de Fonction
function getSum(num1, num2) {
    let total = num1 + num2;
    return total;
}
document.write(getSum(2, 3) + "<br>"); // Prints: 5

// Expression de Fonction
let getSum = function(num1, num2) {
    let total = num1 + num2;
    return total;
}

document.write(getSum(4, 6)); // Prints: 10
```

Une fois que l'expression de la fonction a été stockée dans une variable, la variable peut être utilisée comme une fonction

```
let getSum = function(num1, num2) {  
    let total = num1 + num2;  
    return total;  
}  
  
document.write(getSum(5, 10) + "<br>"); // Prints: 15  
  
let sum = getSum(7, 25);  
document.write(sum); // Prints: 32
```

Remarque : Il n'est pas nécessaire de mettre un point-virgule après l'accolade fermante dans une déclaration de fonction. Mais les expressions de fonction, en revanche, doivent toujours se terminer par un point-virgule.

Conseil : dans JavaScript, les fonctions peuvent être stockées dans des variables, transmises à d'autres fonctions en tant qu'arguments, transmises à des fonctions en tant que valeurs de retour et construites au moment de l'exécution.

Comprendre la portée variable

Vous pouvez déclarer les variables n'importe où dans JavaScript. Mais, l'emplacement de la déclaration détermine l'étendue de la disponibilité d'une variable dans le programme JavaScript, c'est-à-dire l'endroit où la variable peut être utilisée ou accessible. Cette accessibilité est connue sous le nom de portée variable .

Par défaut, les variables déclarées dans une fonction ont une portée locale , ce qui signifie qu'elles ne peuvent pas être visualisées ou manipulées depuis l'extérieur de cette fonction, comme illustré dans l'exemple ci-dessous :

```
// Defining function
function greetWorld() {
    let greet = "Hello World!";
    document.write(greet);
}

greetWorld(); // Prints: Hello World!

document.write(greet); // Uncaught ReferenceError: greet is not defined
```

Cependant, toute variable déclarée dans un programme en dehors d'une fonction a une portée globale , c'est-à-dire qu'elle sera disponible pour tous les scripts, que ce script soit à l'intérieur d'une fonction ou à l'extérieur. Voici un exemple

```
var greet = "Hello World!";

// Defining function
function greetWorld() {
    document.write(greet);
}

greetWorld(); // Prints: Hello World!
document.write("<br>");
document.write(greet); // Prints: Hello World!
```

Fonctions fléchées

Les fonctions fléchées permettent une syntaxe courte pour écrire des expressions de fonction.

Vous n'avez pas besoin du mot-clé **function**, du mot-clé **return** et des **accolades**

```
const x = (x, y) => x * y;  
document.getElementById("demo").innerHTML = x(5, 5);
```

L'utilisation **const** est plus sûre que l'utilisation de **var**, car une expression de fonction est toujours une valeur constante.

Vous ne pouvez omettre le mot-clé **return** et les **accolades** que si **la fonction est une instruction unique**.

Pour cette raison, ce pourrait être une bonne habitude de toujours les garder

```
const x = (x, y) => { return x * y };  
document.getElementById("demo").innerHTML = x(5, 5);
```